CASE-STUDY PRACTICE | CAPSTONE 3 | STUDY EDITION

Video Recommendation: Retrieve Cheaply, Rank Carefully

Scenario questions on a two-stage recommender for a corpus of millions of videos, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on data, a model or a serving fleet: figures are the paper's or arithmetic from stated inputs, and the corpus size, request rate, input widths and hardware speeds are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the problem: the objective, the labels and the funnel 6

2 Data and labels: examples, leakage and freshness 10

3 Candidate generation: classification among millions and fast retrieval 15

4 Ranking, evaluation and operations 21

Sources and confidence 26

Every question, by part

Part 1 Framing the problem: the objective, the labels and the funnel 6

01 Rank by click, or by watch time? Objective 6

02 What does the weighted loss learn? Calculation 7

03 Does a watch-time weight favor long videos? Risk 8

04 Size the funnel Estimation 9

Part 2 Data and labels: examples, leakage and freshness 10

05 A random held-out watch, or the next watch? Labels 10

06 The model that copies the last search Labels 11

07 Teach the model what time it is Freshness 12

08 Review the example builder Code review 13

Part 3 Candidate generation: classification among millions and fast retrieval 15

09 What does a sampled softmax save? Training cost 15

10 Why not a hierarchical softmax? Design choice 16

11 Where do the parameters live? Model design 17

12 Review the retrieval function Code review 18

13 Retrieve from 10 million videos in tens of milliseconds Serving 19

Part 4 Ranking, evaluation and operations 21

14 Share the video embedding across features Features 21

15 Where does the ranking CPU go? Cost 22

16 Fresh signals and churn Operations 23

17 Read the offline numbers, then decide Evaluation 24

18 Ten times the traffic Scale 25

The case in one page

A summary of the case study to read before the questions.

Problem	Recommend videos to over a billion users from a corpus of millions, with a corpus that changes every hour and with labels that are noisy proxies for satisfaction.
Core idea	Two networks. Candidate generation narrows millions of videos to hundreds with coarse features. Ranking scores only those with a rich feature set and sorts by expected watch time.
Candidate generation	Predict the next watched video among millions as a classification, trained with several thousand sampled negatives, more than 100 times faster than a full softmax (source-confirmed).
Ranking	Logistic regression with clicked impressions weighted by watch time and unclicked by 1. The learned odds approximate expected watch time (source-confirmed).
Label choices	Predict the next watch, not a random held-out one. Train on all watches. Add example age and set it to zero when serving (source-confirmed).
Results the paper gives	Ranking loss 41.6% with no hidden layers to 34.6% with a 1024, 512, 256 tower. Equal weighting raises it by 4.1% (source-confirmed).
First to break	Retrieval and ranking compute. An exact scan of 10 million 256-dimensional videos reads 10.24 GB per request and costs about 3 times the ranking stage (illustrative).
Easy-to-miss traps	Ranking is weighted classification. The ranking network over the whole corpus would need millions of cores. A random held-out label leaks the future. A raw watch-time weight can favor long videos.

Numbers to know

Each figure carries the tag that says how it was obtained. The corpus size, request rate, layer widths and hardware speeds are illustrative, because the paper gives scale only in qualitative terms.

Quantity	Value	Evidence
Scale statements	More than a billion users; millions of videos to hundreds to dozens; about one billion parameters; hundreds of billions of examples; tens of milliseconds	source-confirmed (the paper)
Funnel and rate	10,000,000 ÷ 500 = 20,000; 500 ÷ 20 = 25; 2,000,000,000 ÷ 86,400 = 23,148 requests/s; peak 69,444	illustrative; recomputed
Ranking tower	1,000×1,024 + 1,024×512 + 512×256 = 1,679,360 MACs per candidate; 840 million per request; about 972 cores at 20 billion MACs/s	illustrative; recomputed
Where ranking cost sits	Layer 1 61.0%, layer 2 31.2%, layer 3 7.8%; dropping layer 3 saves about 76 cores	illustrative; recomputed
Exact retrieval	10,000,000 × 256 × 4 B = 10.24 GB per request; 2.56 billion MACs; 59 trillion MACs/s at 23,148/s	illustrative; recomputed
Ranking over the corpus	1.68 million × 10 million = 16.8 trillion MACs per request; about 19 million cores	illustrative; recomputed
Sampling cost	Full softmax 1,000,000 × 256 = 256 million MACs; 5,000 sampled = 1.28 million (200 times); paper says more than 100 times	source-confirmed (paper); recomputed
Hierarchical softmax	log2(1,000,000) = 20 nodes × 256 = 5,120 MACs, but the paper could not match accuracy	recomputed
Embedding parameters	3 tables × 1,000,000 × 256 = 768 million (about 3.1 GB); tower 1.19 million (0.15%)	illustrative; recomputed
Embedding against layer	1,000,000 × 32 = 32,000,000 against 2,048 × 2,048 = 4,194,304: 7.6 times; paper says 7 times	source-confirmed (paper); recomputed
Weighted odds	1,000 impressions, 20 clicks of 5 minutes: 100 ÷ 980 = 0.1020; E[T] = 0.1000; E[T] × 1.02 = 0.1020	illustrative; recomputed
Table 1	None 41.6%; 256 36.9%; 512 36.7%; 1024 35.8%; 512-256 35.2%; 1024-512 34.7%; 1024-512-256 34.6%	source-confirmed (the paper)
Ablations	Without powers of continuous features +0.2% loss; equal weighting of positives and negatives +4.1%	source-confirmed (the paper)
Click against watch time	A: 10% × 20 s = 2.0 s; B: 4% × 360 s = 14.4 s per impression	illustrative; recomputed
Example age bias	3,000 watches today over a 28-day window: 107 a day, about 28 times too low	illustrative; recomputed
Exact scan machines	10.24 GB at 100 GB/s = 0.10 s; about 2,400 machines; an index visiting 1%: about 24	illustrative; recomputed
Ten times the traffic	231,481 requests/s average; about 9,700 ranking cores; exact scan 590 trillion MACs/s	illustrative; recomputed

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Ranking objective	Expected watch time per impression	Click probability if clickbait is not a problem	source-confirmed; reasoned
Label	Rolled-back next watch	A random holdout only if future information cannot leak	source-confirmed
Training examples	All watches, a fixed number per user	Recommended watches only if exploration is not a concern	source-confirmed
Softmax training	Candidate sampling with importance weights	Hierarchical softmax if it matches accuracy on your data	source-confirmed
Retrieval	Approximate nearest neighbors in dot product space	An exact scan for a small corpus	source-confirmed; reasoned
Candidate count	Hundreds	More if retrieval recall is the measured limit and ranking CPU allows	reasoned
Embeddings	Shared across features of one ID space	Separate if features must not interact	source-confirmed
Final judgment	Live A/B tests	Offline metrics alone for fast iteration only	source-confirmed

PART 1

Framing the problem: the objective, the labels and the funnel

Choosing what to optimize, reading what a weighted loss actually learns, and sizing the funnel from the corpus to the screen.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Implicit and explicit feedback	Explicit feedback is what users say, such as a thumbs up or a rating. Implicit feedback is inferred from what they do, such as watching a video to the end. It is far more plentiful and far noisier.
Click-through rate and watch time	Click-through rate is the share of impressions that are clicked. Watch time is how long the user then watches. A click says a thumbnail worked, and watch time says whether the video did.
Impression	One video shown to one user in one position on a page. An impression is positive if it was clicked and negative if not.
Odds	For a probability p, the odds are p divided by (1 minus p). Logistic regression with a weighted loss learns the ratio of the total weight on positives to the total weight on negatives.
Weighted logistic regression	Logistic regression in which each example's loss is multiplied by a weight. Here a clicked impression is weighted by its watch time and an unclicked one by 1.
Candidate generation and ranking	Two stages. The first retrieves a few hundred videos from millions using coarse features. The second scores only those with a rich feature set and sorts them.
Multiply-add (MAC)	One multiplication and one addition, the unit used to count the work of a neural network. A layer with a inputs and b outputs costs a times b MACs per example.
Recall ceiling	A two-stage system can never show a video the first stage did not retrieve, so the first stage's recall limits the best result the second stage can reach.
Where to read	Covington, Adams, Sargin: Deep Neural Networks for YouTube Recommendations (RecSys 2016)

01 Rank by click, or by watch time?

OBJECTIVE

SCENARIO

A product manager wants the ranker to maximize click-through rate, since clicks are easy to count. The paper says that ranking by click-through rate often promotes deceptive videos that users do not complete, and that watch time better captures engagement. Make the case with numbers.

HOW TO THINK ABOUT IT

Compute the expected watch time per impression for two videos with different click rates. Then ask what the click metric rewards that the product does not want.

Q What does each metric say about two videos?

A Video A: 10% of impressions are clicked, and a click is watched for 20 seconds on average. Video B: 4% are clicked, and a click is watched for 360 seconds (illustrative).

- Click rate puts A first by 2.5 times. Expected watch time per impression is 0.10 × 20 = 2.0 s for A and 0.04 × 360 = 14.4 s for B, which puts B first by 7.2 times.

Q What is the click metric rewarding?

A Whatever makes the thumbnail and title attractive, whether or not the video delivers. The paper's word for the failure is clickbait (source-confirmed).

- A model trained on clicks learns to produce more of what gets clicked, so the problem grows.

Q How does the paper change the objective?

A Ranking stays a classification of impressions into clicked and unclicked, but each clicked impression is weighted by its watch time. The score then approximates expected watch time (source-confirmed).

- It also says the final objective is constantly tuned with live A/B tests and is generally a simple function of expected watch time per impression.

Q What would you check before adopting it?

A That the new objective raises the outcome you care about in a live test, and that it does not push something else down, for example the share of videos watched to the end, or complaints.

- The paper's evaluation policy is that final judgment comes from A/B tests (Question 17).

Q Reframe it.

A Pick the target that matches the outcome the product wants, then check what the easy-to-count metric rewards that you do not want.

Basis: The clickbait statement and the ranking objective are from the paper (source-confirmed); the two videos and their numbers are illustrative; arithmetic recomputed.

02 What does the weighted loss learn?

CALCULATION

SCENARIO

The paper weights each clicked impression by its watch time T and each unclicked one by 1, then uses the exponential of the model's raw score at inference. An interviewer asks you to show why that score estimates expected watch time. Use 1,000 impressions with 20 clicks, each clicked impression watched for 5 minutes.

HOW TO THINK ABOUT IT

Work out what a logistic regression learns when examples carry weights, then compare with the expected watch time per impression.

Q What does weighted logistic regression learn?

A The odds: the total weight on positive examples divided by the total weight on negative ones. With N examples, k positive, and watch times T_i, that is the sum of the T_i divided by $N - k$ (source-confirmed).

- Without weights the odds would be $k \div (N - k)$, a click odds. The weights replace the count of clicks by the minutes watched.

Q Compute it for the example.

A Weight on positives: $20 \times 5 = 100$ minutes. Weight on negatives: $980 \times 1 = 980$. Odds $= 100 \div 980 = 0.1020$.

- Expected watch time per impression is $100 \div 1{,}000 = 0.1000$ minutes. The odds are slightly larger.

Q What is the approximation?

A Odds $= E[T] \div (1 - P)$, where P is the click probability. For small P that is close to $E[T] \times (1 + P)$, which is the paper's form (source-confirmed).

- Here $P = 0.02$: $E[T] \times 1.02 = 0.1020$, matching the odds. The paper's statement is that, because P is small, the product is close to E[T].

Q When does the approximation weaken?

A When P is not small. At P = 0.20 the odds are E[T] ÷ 0.8 = 1.25 × E[T], and the paper's form gives 1.20 × E[T], a 4% gap.

- For ranking only the order matters, so a smooth bias that affects all videos alike is harmless. A bias that grows with P would favor videos with high click rates.

Q Why the exponential at inference?

A Logistic regression's raw score x is the log of the odds, so exp(x) recovers the odds, which estimate expected watch time. The output is in minutes per impression and is not a probability.

- This is why the paper says ranking is not trained to predict click probability.

Q Reframe it.

A When a loss is modified, work out what the minimizer now estimates. Weighting turned a click odds into a watch-time odds, and the exponential reads it out.

Basis: The weighting, the odds formula and the inference activation are from the paper (source-confirmed); the 1,000 impressions, 20 clicks and 5 minutes are illustrative; the algebra is derived by hand.

03 Does a watch-time weight favor long videos?

RISK

SCENARIO

A reviewer notes that a watch-time weight gives a clicked impression of a 60-minute video that is watched to the end a weight of 3,600 seconds, and a 3-minute video watched to the end 180. The paper does not discuss this. Is it a problem, and what would you do?

HOW TO THINK ABOUT IT

Compute the ratio, then separate what the paper says from what it leaves open.

Q How large is the difference?

A 3,600 ÷ 180 = 20 times, for two videos that were both watched to the end.

- A model trained on these weights has 20 times more to gain by getting the long video right.

Q What does the paper say?

A It says the final ranking objective is generally a simple function of expected watch time per impression and is constantly tuned with A/B tests (source-confirmed).

- A raw sum of watch time is therefore not necessarily what ships. The paper gives no detail on the function.

Q Why might length bias still matter?

A The model learns that long videos carry more weight, so it may push them up even when a short video would satisfy the user better.

- That is reasoning from the weight, not a result in the paper.

Q What would you measure and try?

A Slice results by video length and compare the share of impressions that go to long videos against the share of satisfied sessions.

- Options to test live: cap the weight, weight by the share of the video watched, or add length as a feature. Each changes what the model rewards, so judge them by A/B tests.

Q Reframe it.

A Ask what the unit of your target does when the item changes size. Raw time rewards long items, so check by slice before trusting a total.

Basis: The objective statement is from the paper (source-confirmed); the 60 and 3 minute videos are illustrative; the bias argument and the remedies are reasoned and were not tested.

04 Size the funnel

ESTIMATION

SCENARIO

The paper's Figure 2 shows the funnel from millions of videos to hundreds of candidates to dozens shown, and says only that. A product owner asks how many candidates to keep, and what that does to cost. Use 1 billion users, 2 page loads each a day, 10 million videos, 500 candidates and 20 shown.

HOW TO THINK ABOUT IT

Convert the labels into ratios and rates, then find which stage a change in the candidate count would hit.

Q What are the ratios and the rate?

A 10,000,000 ÷ 500 = 20,000 times fewer after candidate generation. 500 ÷ 20 = 25 times fewer after ranking. In all, 500,000 times.

- 2,000,000,000 loads a day ÷ 86,400 = 23,148 requests a second on average, and 69,444 at an assumed peak of 3 times.

Q Which stage does the candidate count affect?

A Ranking, which runs once per candidate. A ranking tower of 1,000 inputs and 1024, 512 and 256 units costs about 1.68 million MACs per candidate (input width assumed).

- At 500 candidates that is 840 million MACs per request, 19.4 trillion a second, about 972 cores at an assumed 20 billion MACs a second per core.

Q What does doubling the candidates cost?

A Doubling to 1,000 doubles ranking: about 1,944 cores on average, and about 5,800 at peak.

- It raises the recall of the first stage and so the ceiling on quality.

Q What is the ceiling?

A Ranking can only order what candidate generation retrieved. A video missed in the first stage is lost, however good the ranker is.

- Measure the first stage's recall: of the videos the ranker would put on the screen, how many were in the 500.

Q Reframe it.

A In a funnel, the width of each stage sets the cost of the next one and the ceiling of the last. Size the candidate count from cost and recall together.

Basis: The funnel's qualitative shape is from the paper (source-confirmed); the 10 million videos, 500, 20, 2 loads, the peak ratio, the input width and the core speed are illustrative; arithmetic recomputed.

PART 2

Data and labels: examples, leakage and freshness

Which watches to train on, how to build a label that does not leak the future, what to hide from the model, and how to tell it about time.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Surrogate problem	The training problem you actually pose, as a stand-in for the real goal. Here the real goal is a satisfying homepage, and the surrogate is to predict the next watch.
Data leakage	Giving the model information at training time that will not exist at serving time. A model that sees later watches when predicting an earlier one cannot reproduce that at serving.
Held-out watch and rolled-back history	A held-out watch is a random watch removed from the history and predicted from all the others, including later ones. A rolled-back history uses a watch as the label and only the actions before it as input.
Fixed examples per user	Generating the same number of training examples for every user, so a heavy user does not contribute hundreds of times more than a light one.
Example age	How old a training example is, measured back from the end of the training data. It is fed as a feature in training and set to zero when serving.
Training window	The span of history the model is trained on. The paper's is several weeks. A model without a time feature learns the average over the whole window.
Bag of tokens	A set of words or queries with no order. Using one for search history hides which search came last.
Cold start	A video uploaded after the last training run has no watches in the data, so the model has nothing to learn from until the next run.
Where to read	Covington, Adams, Sargin: Deep Neural Networks for YouTube Recommendations (RecSys 2016)

05 A random held-out watch, or the next watch?

LABELS

SCENARIO

A colleague builds training labels the way many collaborative filtering systems do: remove one watch at random from a user's history and predict it from the others. The paper says predicting the next watch worked much better in live tests. Explain why, using a user who watches a series in order.

HOW TO THINK ABOUT IT

Work through one user's history with each scheme, and ask what the model will have available when it serves.

Q What does the random holdout do to the series?

A A user watched episodes 1, 2, 3, 4 and 5 in that order. The holdout removes episode 3 and the model sees episodes 1, 2, 4 and 5.

- It learns that having seen 4 and 5 predicts 3. The paper says this leaks future information (source-confirmed).

Q Why can't that work at serving time?

A At serving time the user has watched episodes 1 and 2 and not yet 4 or 5. The signal the model learned is not available.

- The model also ignores that consumption is asymmetric: episodes are watched in order, and users discover a genre through its popular items before niche ones.

Q What does the rollback do?

A It chooses a random watch as the label and uses only actions before it as input. Episode 3 is predicted from episodes 1 and 2 alone (source-confirmed).

- The training example looks like a serving request: past in, next out.

Q Why is this hard to see in offline tests?

A The paper says the choice has an outsized effect on live A/B results and is very difficult to measure offline.

- An offline test that uses the same leaky split rewards the leak. Check that an offline split mirrors the serving moment before trusting it.

Q Reframe it.

A Build every training example to look exactly like a serving request, and give it only what the model will have at that moment.

Basis: The finding, the two schemes and the consumption pattern are from the paper (source-confirmed); the five-episode user is illustrative.

06 The model that copies the last search

LABELS

SCENARIO

A user has just searched for a singer, and the model must predict the next video. A first version scores well offline and then performs poorly on the homepage. The paper gives this exact example. Find the cause and fix it.

HOW TO THINK ABOUT IT

Ask what the model can learn from knowing the most recent search, and where the label came from.

Q What does the model learn?

A The next video watched after a search is most likely one on that search's results page, so the model learns to reproduce the results page.

- The paper says reproducing the user's last search page as homepage recommendations performs very poorly (source-confirmed).

Q Why does it look good offline?

A Offline labels were generated by the same site behavior, so the shortcut predicts them well. It exploits how the label was produced and not what the user wants.

- The paper's phrase is that the model overfits the surrogate problem.

Q What is the fix?

A Discard the order of searches and represent them as an unordered bag of tokens, so the classifier is no longer directly aware of the origin of the label.

- The search words stay as a signal of interest, and the specific most recent query is hidden.

Q Would a bigger model help?

A No. The problem is what the model is told, not its capacity. More depth would only learn the shortcut better.

- The paper says great care must be taken to withhold information from the classifier.

Q What should you review in other features?

A For each input, ask whether it reveals where the label came from, for example the last page viewed, the slot clicked or the previous recommendation list.

- Remove or blur any input that does.

Q Reframe it.

A Before adding an input, ask whether it carries the answer by another route. A good score offline can come from a shortcut, and withholding is a design tool.

Basis: The example, the failure and the fix are from the paper (source-confirmed); the review checklist is reasoned.

07 Teach the model what time it is

FRESHNESS

SCENARIO

Fresh uploads are important to the product, but a model trained on weeks of history underrates a video uploaded yesterday. The paper feeds the age of each example as a feature and sets it to zero at serving. Explain the bias, the fix and the trap.

HOW TO THINK ABOUT IT

Use a number for how much a new video is underrated, then ask what zero means and when it stops being true.

Q What is the bias?

A The paper says the model reflects the average watch likelihood over the training window of several weeks, and popularity is highly non-stationary (source-confirmed).

- Example (illustrative): a 28-day window. A video uploaded yesterday gets 3,000 watches today. Averaged over the window it scores 3,000 ÷ 28 = 107 a day, about 28 times too low for today.

Q What does example age do?

A It tells the model how long before the end of the training data each example happened, so it can learn a popularity curve: a spike after upload, then a decay.

- Figure 4 shows a model with the feature following a video's upload time and popularity, and a model without it predicting about the window average.

Q Why zero at serving?

A To say that the model is predicting at the very end of the training window. The paper says it sets the feature to zero or slightly negative.

- The paper reports improved offline holdout precision and a dramatic increase in watch time on recently uploaded videos in A/B tests (source-confirmed).

Q What is the trap?

A Zero means that now is the end of the window. If the served model is old, now is later than that, and the model understates how fresh things are.

- That follows from the design, not from a result in the paper. Retrain often, and monitor how far the deployed model is from its window's end.

Q What does it not fix?

A A video uploaded after the last training run has no watches in the data and no embedding. The feature cannot help until the next run.

- That is a separate cold-start problem, reasoned from the design.

Q Reframe it.

A A model trained on the past will describe the past. Give it the time of each example and tell it what time it is when you ask it a question.

Basis: The bias, the feature, the serving value and the A/B outcome are from the paper (source-confirmed); the 28-day window and 3,000 watches are illustrative arithmetic; the staleness trap and cold start are reasoned.

08 Review the example builder

CODE REVIEW

SCENARIO

A colleague writes the function below to build training examples from a user's watch history, and the training loss looks excellent. In a live test the recommendations are poor, and heavy users seem to be served much better than light ones. Find the faults.

```
def build_examples(history):
    # history: list of (timestamp, video_id), oldest first
    examples = []
    for i, (t, label) in enumerate(history):
        context = [v for j, (_, v) in enumerate(history) if j != i]
        examples.append((context, label))
    return examples
```

HOW TO THINK ABOUT IT

Compare each line with the paper's three choices: roll back, a fixed number of examples per user, and example age.

Q What is wrong with the context?

A For each watch it uses every other watch as input, including the ones that came later, so the model sees the future (Question 5).

- This explains the excellent training loss and the poor live result.

Q What is wrong with the count of examples?

A It creates one example for every watch. A user with 1,000 watches gives 1,000 examples and one with 10 gives 10, so heavy users carry 100 times the weight.

- The paper generates a fixed number of examples per user to weight users equally (source-confirmed).

Q What is missing?

A Example age. Without it the model learns the average over the window and underrates fresh videos (Question 7).

- It is the label's time subtracted from the latest time in the training data.

Q What else is wrong?

A It rebuilds the context list for every watch, which takes time that grows with the square of the history length.

- Slicing history up to the label's position is linear.

Q State the fix.

A Pick a fixed number of random label positions per user, use only history before each, and attach the age of the label.

- Test it with a user whose history has an obvious order, and check that no input is later than its label.

Q Reframe it.

A Check a data pipeline for leakage and for who dominates the loss before looking at the model. Loss that is too good is a symptom to investigate.

REFERENCE CODE

```
import random

def build_examples(history, t_max, per_user=5, rng=random):
    # history: list of (timestamp, video_id), oldest first
    if len(history) < 2:
        return []
    examples = []
    for _ in range(per_user):
        i = rng.randrange(1, len(history))        # label position
        t, label = history[i]
        context = [v for _, v in history[:i]]     # only actions before the label
        examples.append((context, label, t_max - t))   # age = t_max - t
    return examples
```

Basis: The three design choices are from the paper (source-confirmed); the 1,000 and 10 watch users are illustrative; the fix was checked by hand and not run on data.

PART 3

Candidate generation: classification among millions and fast retrieval

Why the first stage is a classifier, what a sampled softmax saves, what the network holds, and how to retrieve from millions of videos in tens of milliseconds.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Extreme multiclass classification	Classification where the number of classes is huge. Here each video is a class, and the model predicts which video the user will watch next among millions.
Softmax	A function that turns scores into probabilities by exponentiating each and dividing by the sum over all classes. With millions of classes the sum is the expensive part.
Candidate sampling (sampled softmax)	Computing the loss over the true class and a few thousand sampled negative classes only, with an importance-weight correction for the sampling.
Hierarchical softmax	Arranging the classes in a tree so that one choice among millions becomes a path of binary choices. It is cheap per example, but each node must separate groups of classes that may be unrelated.
Embedding table	A lookup from an ID to a learned vector. Most parameters of these models are in embedding tables, not in the hidden layers.
Nearest-neighbor search in dot-product space	Finding the stored vectors with the largest dot product with the query vector. Exact search compares against all of them, and approximate search compares against a part.
Recall of an approximate index	The share of the true top results that the approximate search also returns. It measures what the speed-up cost.
Memory bandwidth	How many bytes a machine can read from memory each second. A scan that must read every vector cannot run faster than the data size divided by this.
Where to read	Covington, Adams, Sargin: Deep Neural Networks for YouTube Recommendations (RecSys 2016)

09 What does a sampled softmax save?

TRAINING COST

SCENARIO

Candidate generation predicts the next watch among millions of videos, so an ordinary softmax sums over every video for every training example. The paper samples several thousand negatives instead and reports a speedup of more than 100 times. Check that claim with your own arithmetic.

HOW TO THINK ABOUT IT

Count the multiply-adds in the output layer for each approach, then ask what the paper states and what you assumed.

Q What does the full softmax cost?

A The output layer scores every class: 1,000,000 videos × 256 dimensions = 256 million MACs per example (the paper's experiments use 1 million classes and 256 dimensions).

- Over 200 billion examples (illustrative, within the paper's hundreds of billions) that is 5.1 × 10^19 MACs for one forward pass of the output layer.

Q What does the sampled version cost?

A 5,000 sampled negatives × 256 = 1.28 million MACs per example, 200 times fewer. The same 200 billion examples take 2.6 × 10^17 MACs.

- At an assumed 10^12 MACs a second, that is about 593 days against 3 days on one accelerator, for the output layer alone.

Q What does the paper actually say?

A In practice several thousand negatives are sampled, which corresponds to more than 100 times speedup over a traditional softmax (source-confirmed).

- The 5,000 is a value inside its range and the 200 times is arithmetic, so the two are consistent without 200 being a quotation. With a vocabulary of millions the ratio is larger.

Q What does sampling change in the math?

A The loss is computed over the true class and the negatives only, so it is a biased estimate. The paper corrects it with importance weighting (source-confirmed).

- The sampled negatives come from the background distribution of videos, so popular videos are sampled more.

Q Reframe it.

A When a sum is over a huge set, estimate it from a sample, and correct for how the sample was chosen. Then check your speedup by counting operations.

Basis: The sampling method and the more-than-100-times claim are from the paper (source-confirmed); the 200 billion examples, 5,000 negatives and 10^12 MACs a second are illustrative; arithmetic recomputed.

10 Why not a hierarchical softmax?

DESIGN CHOICE

SCENARIO

A teammate points out that a hierarchical softmax is much cheaper than sampling: it needs only about 20 binary decisions for a million classes. The paper tried it and went with sampling. Explain why a cheaper method lost.

HOW TO THINK ABOUT IT

Count the cost, and then ask what the cost does not capture.

Q How cheap is the tree?

A A balanced binary tree over 1,000,000 classes has depth log2(1,000,000) = about 20. At 256 dimensions per node, 20 × 256 = 5,120 MACs per example.

- That is about 250 times cheaper than the sampled softmax's 1.28 million MACs, and about 50,000 times cheaper than the full softmax.

Q So why did the paper not use it?

A Accuracy. The paper says it was not able to achieve comparable accuracy with hierarchical softmax (source-confirmed).

- Cost was not the objection, so a cost comparison alone cannot settle the choice.

Q What is the stated reason?

A Each node in the tree has to discriminate between sets of classes that are often unrelated, which makes the classification problem much more difficult and degrades performance.

- A leaf's path depends on how videos were grouped, and any grouping puts unlike videos together.

Q What would you do if you had to decide yourself?

A Compare both on the quality metric you ship on, under the same budget, and include the cost of building and updating the tree as the corpus changes.

- The paper's result is for its data. Treat it as a strong prior, not a law.

Q Reframe it.

A The cheapest method is not the best one if it loses on the quality metric. Compare on the metric that matters, and treat cost as a constraint.

Basis: The rejection and its reason are from the paper (source-confirmed); the tree depth and MAC counts are arithmetic (recomputed) on the 256-dimension, 1 million class setting.

11 Where do the parameters live?

MODEL DESIGN

SCENARIO

The candidate generation network averages embeddings of watched videos and search tokens, concatenates demographics, and feeds ReLU layers. A manager asks what to cut if the model must be smaller. Use the paper's experimental configuration.

HOW TO THINK ABOUT IT

Count parameters in the embedding tables and in the layers before deciding where the size is.

Q What is the architecture?

A Watched video IDs and search tokens (unigrams and bigrams) are mapped through learned embeddings and averaged. Demographics and example age are added. Fully connected ReLU layers follow (source-confirmed).

- Averaging performed best among several options, including sum and component-wise max. The experiments used at most 50 recent watches and 50 recent searches.

Q How many parameters are in the embeddings?

A 1,000,000 videos, 1,000,000 search tokens and 1,000,000 output classes at 256 dimensions: 3 × 256,000,000 = 768 million parameters, about 3.1 GB in float32.

- The paper says its production models learn approximately one billion parameters. The two are the same order. The paper does not give the experimental models' size.

Q How many are in the layers?

A For the 1024, 512, 256 tower with an assumed input width of 520: 520 × 1,024 + 1,024 × 512 + 512 × 256 = 1.19 million weights.

- That is 0.15% of the embedding parameters. Cutting layers barely reduces size.

Q What does depth buy?

A Depth lets the network model interactions between features. Depth zero is effectively a linear factorization and performed very similarly to the predecessor system (source-confirmed).

- Figure 6 shows that features and depth each improved holdout MAP. The paper gives no values in the text.

Q What would you cut?

A The embedding vocabularies: fewer video IDs or search tokens, or lower dimensions. That is where the size is, and each choice costs accuracy that must be measured.

- Savings in layers would help compute, not memory.

Q Reframe it.

A Count before cutting. In models with large ID spaces the table, not the network, holds nearly all the parameters.

Basis: The architecture, the configuration and the depth findings are from the paper (source-confirmed); the input width of 520 is illustrative; the parameter counts are arithmetic (recomputed).

12 Review the retrieval function

CODE REVIEW

SCENARIO

A colleague writes the retrieval step below and tests it on 10,000 videos, where it works. In staging, with 10 million videos, some requests return garbage and the rest are slow. The paper says calibrated probabilities are not needed at serving time. Find the faults.

```
import numpy as np

def generate_candidates(u, video_vecs, video_ids, top_n=500):
    scores = video_vecs @ u
    probs = np.exp(scores) / np.exp(scores).sum()    # softmax over the corpus
    order = np.argsort(-probs)
    return [video_ids[i] for i in order[:top_n]]
```

HOW TO THINK ABOUT IT

Check what the softmax adds, what happens to the exponential on large scores, and what the sort costs at the real size.

Q What does the softmax add?

A Nothing for retrieval. The exponential is strictly increasing, so the order by dot product is the order by probability (source-confirmed that probabilities are not needed).

- Skip it and sort the raw scores.

Q What goes wrong with it at scale?

A The exponential of a large score overflows. In float32 any score above about 88 becomes infinity, and infinity divided by infinity is not a number.

- Those rows give not-a-number probabilities, and the sort puts them in an arbitrary place. This explains the garbage.

Q What is wrong with the sort?

A It sorts all 10 million values to keep 500, which takes time proportional to n log n, when a partition finds the largest 500 in linear time.

- Then sort the 500 only.

Q Is the fixed version enough?

A It fixes correctness and speed on one machine, and it is still an exact scan: 2.56 billion MACs per request and 10.24 GB read.

- At 23,148 requests a second that does not fit (Question 13). It remains useful as the reference to measure an approximate index against.

Q State the fix, and the test.

A Use the raw scores, take the top 500 with a partition, then sort those. Test with scores in the hundreds, ties, and fewer videos than the requested top n.

- Compare against a slow, simple version on a small sample.

Q Reframe it.

A Do not compute what the consumer does not use. An extra step can be harmless at test size and destructive at production size.

REFERENCE CODE

```
import numpy as np

def generate_candidates(u, video_vecs, video_ids, top_n=500):
    scores = video_vecs @ u                          # order by raw dot product
    n = min(top_n, len(scores))
    top = np.argpartition(scores, -n)[-n:]           # linear-time selection
    top = top[np.argsort(-scores[top])]              # sort only the winners
    return [video_ids[i] for i in top]
```

Basis: The statement that calibrated probabilities are not needed at serving time is from the paper (source-confirmed); the overflow threshold is standard float32 behavior; the cost figures are arithmetic (recomputed); the fix was checked by hand and not run.

13 Retrieve from 10 million videos in tens of milliseconds

SERVING

SCENARIO

The paper says scoring millions of items in tens of milliseconds requires an approximate scheme that is sublinear in the number of classes, and that A/B results were not sensitive to the choice of algorithm. You have 10 million videos of 256 float32 values and 23,148 requests a second. Decide how to retrieve.

HOW TO THINK ABOUT IT

Compute the cost of an exact scan on bandwidth and on arithmetic, then see what an approximate index changes and what it risks.

Q What does an exact scan cost?

A 10,000,000 × 256 × 4 bytes = 10.24 GB read per request, and 2.56 billion MACs. At an assumed 100 GB/s that is 0.10 s on one machine, longer than the tens of milliseconds the paper requires.

- One machine serves about 10 requests a second, so 23,148 a second needs about 2,400 machines on average.

Q Does batching help?

A Yes for throughput. One pass over the embeddings can serve a batch of requests, which turns the work from bandwidth-bound to compute-bound.

- It adds waiting time while a batch fills, which spends the latency budget. Total arithmetic stays at 59 trillion MACs a second.

Q What does an approximate index do?

A It visits only a part of the vectors. If it reads 1% of them, that is 102 MB per request, about 1 ms at 100 GB/s, and one machine serves about 970 requests a second, so about 24 machines on average (illustrative).

- The paper says previous systems at YouTube used hashing and the described classifier uses a similar approach, with general purpose nearest-neighbor libraries for the dot product search.

Q What does it risk?

A Missing some of the true top results. Measure recall of the index against the exact scan, and the effect on the ranked list.

- The paper found A/B results not particularly sensitive to the algorithm. Check that on your data. The recall ceiling still applies.

> **Q Reframe it.**
>
> **A** Price the exact method on the hardware before accepting it, then spend an approximation where the measured effect on the product is small.

Basis: The need for a sublinear scheme and the A/B insensitivity are from the paper (source-confirmed); the 100 GB/s, the 1% visit rate and the machine counts are illustrative; arithmetic recomputed.

PART 4

Ranking, evaluation and operations

Features and shared embeddings, where ranking cost sits, fresh signals and churn, reading offline and online evidence, and scaling traffic.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Shared embedding	One embedding table used by several features drawn from the same ID space, for example the video ID of the impression and the last video watched.
Quantile normalization	Rescaling a continuous feature through its cumulative distribution so that it is spread evenly between 0 and 1. The paper also feeds its square and square root.
Churn	Successive page loads not returning identical lists, because features such as how often a video was already shown change between requests.
Weighted, per-user loss	On one user's page, if an unclicked impression scores higher than a clicked one, the clicked impression's watch time counts as mispredicted. The loss is that share of all watch time.
Mean average precision (MAP)	An information retrieval metric that averages precision over the ranked results of many queries. The paper uses it for candidate generation on held-out next watches.
Offline and online evaluation	Offline evaluation scores a model on logged data. Online evaluation measures real users in an A/B test. The paper's policy is that online decides.
Feedback loop	What the system recommends shapes what users watch, which becomes the next training set.
Slice	A subset of requests, for example by video length or country, used to see effects that an overall average hides.
Where to read	Covington, Adams, Sargin: Deep Neural Networks for YouTube Recommendations (RecSys 2016)

14 Share the video embedding across features

FEATURES

SCENARIO

The ranking model uses hundreds of features. Several of them are video IDs: the impression's video, the last video the user watched, and the video that seeded the recommendation. An engineer wants a separate embedding table for each, for flexibility. Decide, using the paper and the parameter arithmetic.

HOW TO THINK ABOUT IT

Count the parameters in a table against a layer, then ask what sharing costs and what keeps features distinct.

Q How big is one embedding table?

A The paper's example: one million IDs in a 32 dimensional space is 32,000,000 parameters, against 2,048 × 2,048 = 4,194,304 in a fully connected layer 2,048 units wide, a ratio of 7.6. The paper says 7 times.

- The overwhelming majority of parameters are in the high-cardinality embedding spaces (source-confirmed).

Q What would separate tables cost?

A With three video ID features, three tables: 96,000,000 parameters in place of 32,000,000, three times the memory for the same IDs.

- Each table also sees only the data of its own feature, so rare videos get fewer updates.

Q What does sharing give?

A The paper says sharing embeddings is important for improving generalization, speeding up training and reducing memory.

- A video learned from one role helps in the others.

Q How do features stay distinct?

A Each feature is still fed separately into the network, so the layers above can learn a specialized representation for each.

- Only the lookup is shared.

Q What other feature details matter?

A Embedding dimension grows roughly with the logarithm of the number of unique values. Large ID spaces keep the top N by frequency in clicked impressions, and unknown IDs map to a zero embedding.

- Continuous features are rescaled by their distribution, with the square and square root added. Removing the powers raised the loss by 0.2% (source-confirmed).

Q Reframe it.

A Share what represents the same thing and keep separate what must differ. Count parameters first to find where the cost is.

Basis: The embedding statements and the 7 times figure are from the paper (source-confirmed); the arithmetic is recomputed; the three-table comparison is illustrative.

15 Where does the ranking CPU go?

COST

SCENARIO

Ranking CPU is the paper's binding constraint: its best tower is the largest one that stays within the serving CPU budget. A manager asks whether dropping the third layer is worth it. Use the Step 2 figures and the paper's Table 1.

HOW TO THINK ABOUT IT

Find how the work splits across layers, then compare what a layer costs with what it buys in the table.

Q How does the work split?

A With an assumed input of 1,000: layer 1 is 1,000 × 1,024 = 1,024,000 MACs (61.0%), layer 2 is 1,024 × 512 = 524,288 (31.2%), layer 3 is 512 × 256 = 131,072 (7.8%). Total 1,679,360.

- The first layer dominates, so the input width matters more than depth.

Q What does the table say about the last layer?

A 1024, 512 gives a weighted per-user loss of 34.7% and adding the 256 layer gives 34.6% (source-confirmed). The layer buys 0.1 points.

- The paper chose the three-layer tower because it gave the best results while staying within the serving CPU budget.

Q What does dropping it save?

A 7.8% of ranking compute: about 76 of the 972 cores needed on average, and about 230 at peak (illustrative core speed).

- Whether 0.1 points is worth 76 cores depends on what 0.1 points is worth in live metrics, which only an A/B test shows.

Q What other levers are there?

A Fewer candidates, a narrower input, or a cheap first pass that drops weak candidates before the full tower.

- Each trades quality for cost. A cheap first pass adds a second recall ceiling, so measure what it drops.

Q Reframe it.

A Price each part of a model in the resource that binds, then compare it with what that part buys. Often one layer takes most of the cost and another buys almost nothing.

Basis: Table 1 values and the CPU-budget statement are from the paper (source-confirmed); the input width of 1,000 and the core speed are illustrative; the MAC counts and core estimates are arithmetic (recomputed).

16 Fresh signals and churn

OPERATIONS

SCENARIO

A feature such as how many times this video was already shown to this user is computed from the last few seconds of activity. A platform engineer proposes refreshing these features once a day, to save infrastructure cost. Say what happens.

HOW TO THINK ABOUT IT

Follow one user through a session of page loads, with each design.

Q What does the feature do in the paper?

A Features describing the frequency of past impressions introduce churn: if a user was recently recommended a video and did not watch it, the model demotes it on the next page load (source-confirmed).

- The paper calls serving up-to-the-second impression and watch history an engineering feat of its own, and vital for responsive recommendations.

Q What happens with a daily refresh?

A A user loads the page 5 times in a session. The impression counts do not change between loads, so the five lists are the same, and the user is shown the same ignored videos again.

- Each reload wastes the slots on videos the user just passed.

Q What else is lost?

A Features such as the time since the user last watched on this topic go stale, so the ranking cannot react to what the user just did.

- The paper says the strongest signals describe the user's previous interaction with the item and similar items.

Q What is the cost of doing it right?

A A store that is updated on every impression and watch event and read on every request, for each candidate scored. That is the cost the engineer wanted to save.

- It must hold up under the request rate of Question 4, and a delay in it shows up as stale features.

Q What would you decide?

A Keep the fast path for the features that cause churn and the user's latest actions, and refresh slow-moving features less often.

- Measure the effect of each feature group's freshness in an A/B test before deciding what to relax.

Q Reframe it.

A Features have a shelf life. Match how often you refresh each one to how quickly it changes the right answer.

Basis: The churn mechanism and the real-time history statement are from the paper (source-confirmed); the five-reload session is illustrative; the design options are reasoned and untested.

17 Read the offline numbers, then decide

EVALUATION

SCENARIO

The paper reports a weighted, per-user loss falling from 41.6% to 34.6% as hidden layers are added, and says that live A/B results are not always correlated with offline ones. A teammate wants to ship the best offline model without a live test. Explain what the numbers say and what they cannot.

HOW TO THINK ABOUT IT

Work through the metric on a tiny example, read the table, then ask what the metric shares with the model being scored.

Q How is the metric computed?

A On a page, if an unclicked impression scores higher than a clicked one, the clicked impression's watch time is mispredicted. The loss is the mispredicted watch time divided by the total (source-confirmed).

- Example: page 1 has a clicked video watched 8 minutes, scored below an unclicked one. Page 2 has a clicked video watched 2 minutes, ranked correctly. Loss = 8 ÷ (8 + 2) = 80%.

Q What does Table 1 show?

A The loss falls from 41.6% with no hidden layers to 34.6% with 1024, 512, 256 units, a drop of 7.0 points or 16.8% relative. Wider and deeper both help, at the cost of server CPU time.

- Equal weighting of positives and negatives raised the loss by 4.1%.

Q What do the numbers share with the model?

A The metric is weighted by watch time, as is the training loss. A model trained on watch-time weights does well on a watch-time metric by design.

- So the 4.1% says that weighting works for that metric. It is not evidence about user satisfaction (reasoned).

Q What does the paper say about offline against online?

A Offline metrics guide development. Final effectiveness is decided in live A/B tests, because live results are not always correlated with offline ones (source-confirmed).

- Its example: the next-watch against random-holdout label choice, which has an outsized effect on A/B results and is very difficult to measure offline.

Q What would you ship on?

A The offline metric to choose candidates to test, and a live test to decide, with a guard on the metrics you do not want to harm.

- For example-age, both agreed. The policy is there for the cases where they do not.

Q Reframe it.

A Ask what the metric shares with the model. Offline numbers steer the search, and a test on real users decides what ships.

Basis: Table 1 values, the metric definition and the evaluation policy are from the paper (source-confirmed); the two-page example is illustrative arithmetic; the point about shared weighting is reasoned.

18 Ten times the traffic

SCALE

SCENARIO

Imagine ten times the users or the page loads. A manager asks what breaks first, and what to add. Use the Step 2 anchors.

HOW TO THINK ABOUT IT

Scale each quantity by ten and sort by what it follows: requests, the corpus, or the vocabulary.

Q What happens to the rate?

A 23,148 requests a second becomes 231,481 on average, and 69,444 becomes 694,444 at peak.

- Every per-request cost scales with it.

Q What happens to ranking?

A About 972 cores become about 9,700 on average and about 29,000 at peak (illustrative core speed).

- It yields to machines. Levers that cut cost per request: a narrower input, fewer candidates, a cheap first pass.

Q What happens to retrieval?

A An exact scan would need about 590 trillion MACs a second and ten times the bandwidth. An approximate index that visits a part scales with the share it visits.

- If the corpus also grows, the scan grows with it, which is why a sublinear index is the lever (Question 13).

Q What does not follow requests?

A The embedding tables follow the vocabulary of videos and tokens, and freshness follows the upload rate. Traffic does not change either.

- Memory per replica stays about the same, while the number of replicas grows with the request rate.

Q Reframe it.

A Sort the limits by what they scale with. Some follow requests and yield to machines. Others follow the corpus, and need a design change.

Basis: The qualitative scale is from the paper (source-confirmed); the 10 times load, the core speed and the results are illustrative; arithmetic recomputed.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #3. The one source is the YouTube RecSys 2016 paper, whose full text was read on 2026-10-17. The paper reports scale only qualitatively (millions of videos, hundreds of candidates, dozens shown), so every concrete size in this book is an illustrative assumption. It describes the models and the labels but not the retrieval index, the serving fleet or the feature store, so the answers about those are design arguments made for this book and were not run on real systems.

- Covington, Adams, Sargin: Deep Neural Networks for YouTube Recommendations, RecSys 2016, full text read 2026-10-17
https://research.google.com/pubs/archive/45530.pdf